The Journal of Open Source Software

‘SRS’ R package and ‘q2-srs’ QIIME 2 plugin: normalization of microbiome data using scaling with ranked subsampling (SRS)

Vitor Heidrich[*1, 2], **Petr Karlovsky**[3], **and Lukas Beule**[†4]

1 Centro de Oncologia Molecular, Hospital Sírio-Libanês, São Paulo, SP, Brazil **2** Departamento de Bioquímica, Instituto de Química, Universidade de São Paulo, São Paulo, SP, Brazil **3** Molecular Phytopathology and Mycotoxin Research, Faculty of Agricultural Sciences, University of Goettingen, Goettingen, Germany **4** Julius Kühn Institute (JKI) – Federal Research Centre for Cultivated Plants, Institute for Ecological Chemistry, Plant Analysis and Stored Product Protection, Berlin, Germany

DOI: 10.21105/joss.03489

Software

- Review
- Repository
- Archive

Editor: Pending Editor

Submitted: 12 July 2021
Published: 12 July 2021

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Several ecological data types, especially microbiome count data, are commonly sample-wise normalized before analysis to correct for sampling bias and other technical artifacts. Recently, we developed an algorithm for normalization of ecological count data called ‘scaling with ranked subsampling’ (SRS), which surpasses widely adopted ‘rarefying’ (random subsampling without replacement) in reproducibility and in safeguarding the original community structure. Here we describe an implementation of the SRS algorithm in the ‘SRS’ R package and the ‘q2-srs’ QIIME 2 plugin. We also provide accessory functions for dataset exploration to guide the choice of parameters for SRS.

Statement of need

High-throughput sequencing of taxonomically informative loci of microbial genomes by amplicon sequencing dramatically improved our understanding of microbial communities. Microbiome research expanded into all microbial habitats on earth, including the human intestine (e.g. Yatsunenko et al. (2012)), soils (e.g. Fierer (2017)) and deep-sea sediments (e.g. Orsi (2018)). A range of bioinformatic tools and platforms as well as reference databases have been developed to enable the extraction of biological insight from the large amounts of data generated by multiplexed amplicon sequencing. The number of sequence counts per sample (sequencing depth) obtained from such sequencing runs can vary by orders of magnitude (McMurdie & Holmes, 2014). Those variations are technical artifacts, which must be computationally compensated by normalization to prevent bias. Therefore, normalization of sequence counts per sample is a key step in processing amplicon sequencing data.

Unequal sampling depth is a problem inherent not only to microbiome research but to all studies based on ecological count data. Therefore, we recently introduced scaling with ranked subsampling (SRS) for the normalization of ecological count data and successfully applied it to microbiome analysis (Beule & Karlovsky, 2020). Yet, the implementation of SRS in bioinformatic platforms was missing.

In this work, we introduce an R package (‘SRS’) and a QIIME 2 plugin (‘q2-srs’) for the normalization of microbiome count data using SRS. Furthermore, we improve the original

*contributed equally, corresponding author
†contributed equally, corresponding author

SRS algorithm and add features to visualize and evaluate the results. Finally, we provide an example for microbial ecologists that aim to normalize microbiome count data obtained by amplicon sequencing.

Principle of SRS

SRS algorithm performs scaling followed by ranked subsampling.

- Scaling: feature counts (such as OTUs (operational taxonomic units), ASVs (amplicon sequence variants) or clades) are scaled sample-wise so that the sum of the scaled counts (C_{scaled}) for all samples are equal to the desired number of counts (C_{min}).
- Ranked subsampling: the scaling step produces fractional values that must be converted into counts (integers). To do this, the C_{scaled} for each feature is split into the floor (C_{int}) and fractional part (C_{frac}) of C_{scaled}. Because $C_{min} = \sum C_{scaled} = \sum C_{int} + \sum C_{frac}$, follows that $C_{min} \geq \sum C_{int}$. Therefore, ΔC C_{frac} values (where $\Delta C = C_{min} - \sum C_{int}$) must be converted into additional counts (integers) so that C_{min} can be reached. To do so, C_{frac} values are ranked. Next, from the highest to the lowest rank, a count for each feature is added until ΔC counts have been added. After this step, all samples will have been normalized to C_{min} counts.
- Special cases: (i) when C_{frac} values involved in picking ΔC counts share the same rank across features, the counts are added for features based on the respective C_{int} ranks; (ii) when both C_{frac} and its respective C_{int} values involved in picking ΔC counts share the same ranks across features, the counts are assigned randomly (without replacement). The specification of the seed that initializes the random process enables reproducible results.

Features

'SRS' R package

`SRS`-function

The SRS algorithm was implemented as the `SRS`-function in the 'SRS' R package. As an extension of the original SRS algorithm published by Beule & Karlovsky (2020), SRS as implemented in version 0.2.2 of the package enables reproducible results in case SRS uses random subsampling without replacement by specifying the seed that initializes the random process (`set.seed`). The default settings of the `SRS`-function (as of version 0.2.2) are:

```
SRS(data, Cmin, set_seed = TRUE, seed = 1)
```

where `data` is the input data (e.g. an OTU table), with samples distributed column-wise, `Cmin` is the number of counts to which all samples will be normalized (C_{min}), `set_seed` enables the use of the `set.seed`-function, and `seed` specifies the seed used by `set.seed` to initialize the random process.

`SRScurve`-function

In analogy to rarefaction curves, the `SRScurve`-function of the 'SRS' R package plots the number of observed unique features (observed richness) against the number of sampled counts utilizing the `SRS`-function (SRS curves). In addition to observed richness, different alpha

DRAFT

diversity metrics (Shannon, Simpson and inverse Simpson indices as implemented in the d iversity-function of the 'vegan' R package (Oksanen et al., 2013)) can be selected to generate SRS curves. Furthermore, SRScurve allows a direct comparison to averaged repeated rarefying. The default settings of the SRScurve-function (as of version 0.2.2) are:

```
SRScurve(data, metric = "richness", step = 50, sample = 0,
   max.sample.size = 0, rarefy.comparison = FALSE,
   rarefy.repeats = 10, rarefy.comparison.legend = FALSE,
   xlab = "sample size", ylab = "richness", label = FALSE,
   col, lty, ...)
```

where data is the input data (e.g. an OTU table), metric selects the alpha diversity metric to be plotted ("richness" = observed richness; "shannon" = Shannon index; "simpson" = Simpson index; "invsimpson" = inverse Simpson index), step specifies the step size at which the alpha diversity metric are sampled, sample specifies the cutoff-level to visualize trade-offs between cutoff-level and alpha diversity, max.sample.size specifies the maximum sample size to which SRS curves are drawn (the default does not limit the maximum sample size), rarefy.comparison enables comparison of SRS curves to rarefying, rarefy.repeats specifies the number of repeats used for rarefying, rarefy.comparison.legend, xlab, ylab, label, col, lty, and ... are graphical parameters.

SRS.shiny.app-function

The SRS.shiny.app-function of the 'SRS' R package launches a Shiny app for SRS in the default web browser to determine C_{min}. The app utilizes the SRScurve-function and enables the selection of four diversity metrics (see metric in SRScurve) that will be returned at different C_{min}. The selection of C_{min} is interactive through a slider or an interconnected numeric text field. In response to the selected C_{min}, the app returns

1. a rug plot that shows the distribution of the number of counts per sample and displays discarded samples as well as summary statistics (including a list of discarded samples and descriptive statistics of the global feature richness and selected alpha diversity metric of the input dataset) in response to the selected C_{min} (Figure 1A),
2. a plot of SRS curves (SRScurve-function) that respond to the selected step size (step) and maximum sample size (max.sample.size) (Figure 1B), and
3. an interactive table with sample names and the number of counts per sample as well as the initial diversity (non-normalized), retained diversity (normalized), %retained diversity (normalized) and %discarded diversity (normalized) of the selected alpha diversity metric in response to the selected C_{min} (Figure 1C).

The default C_{min} of the app is the lowest total number of counts per sample in the input data (no samples are discarded by default), which can be restored within the app using the reset Cmin-button. The default maximum sample size equals the default setting of C_{min} and can be restored using the reset max. sample size-button. The default step size for SRS curves is 1,000. The default setting of the SRS.shiny.app-function (as of version 0.2.2) is:

```
SRS.shiny.app(data)
```

where data is the input data (e.g. an OTU table).

DRAFT

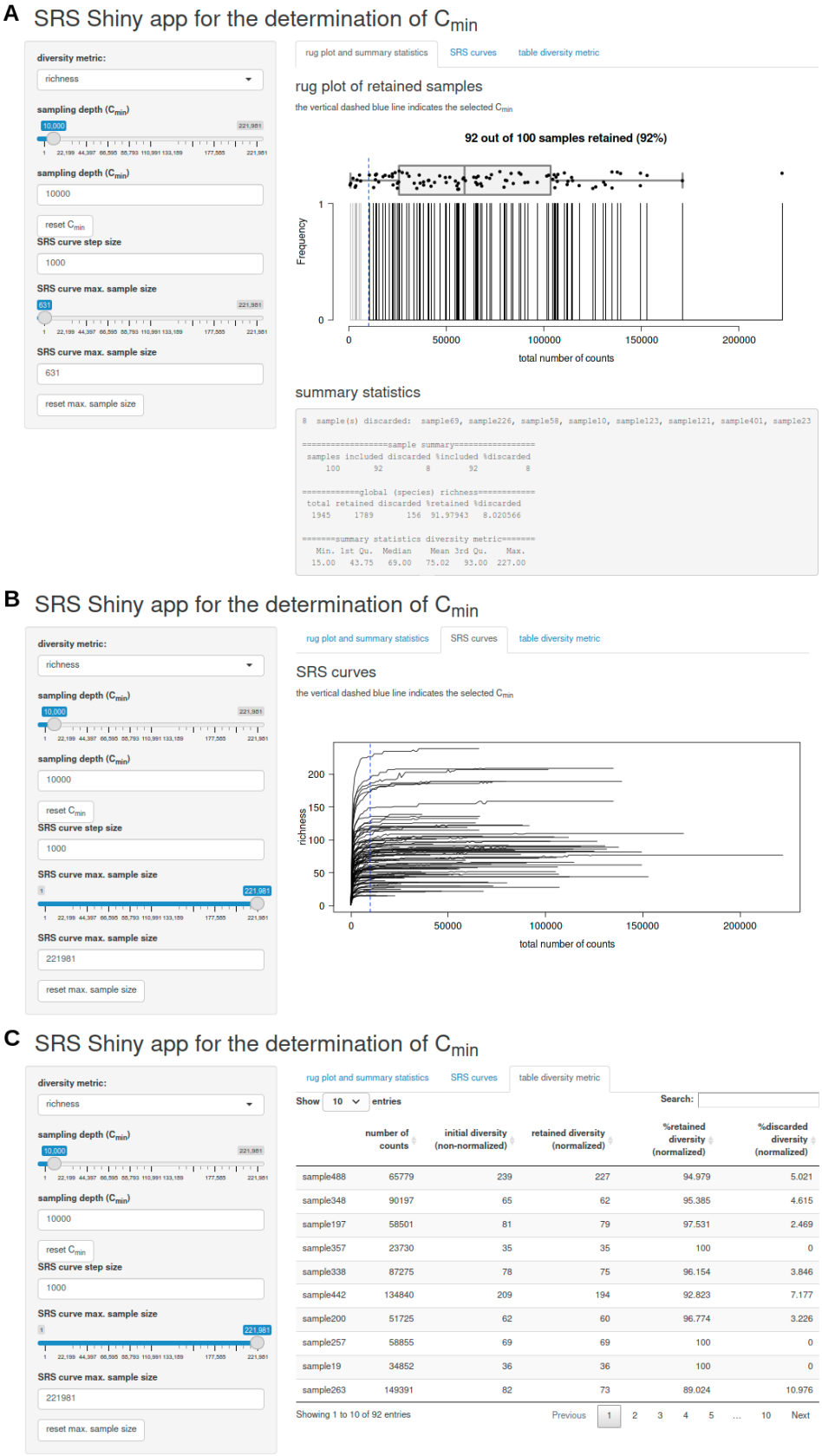

Figure 1: User interface of the Shiny app for SRS (`SRS.shiny.app`-function of the 'SRS' R package version 0.2.2).

‘q2-srs’ QIIME 2 plugin

The ‘q2-srs’ QIIME 2 plugin allows straightforward SRS algorithm incorporation to QIIME 2 pipelines. Because its implementation wraps up the ‘SRS’ R package, its functionalities are analogous to those presented in the previous section.

Specifically, ‘q2-srs’ features the QIIME 2 actions `SRS` and `SRScurve`, which mirror the ‘SRS’ R package SRS-function and SRScurve-function, respectively, with the same behaviour and default parameters as presented in the previous section. The command-line interface commands for the use of the `SRS`- and `SRScurve`-functions within QIIME 2 environment are, respectively: `qiime srs SRS` and `qiime srs SRScurve`. Finally, despite the ‘q2-srs’ QIIME 2 plugin not having a `SRS.shiny.app`-function counterpart, an online version of the SRS Shiny app is provided for ‘q2-srs’ users.

Results and discussion

In both the R package as well as the QIIME 2 plugin, we modified the original SRS algorithm by specifying a seed that initializes the random process (`set.seed`) in cases where the SRS uses random subsampling without replacement of the lowest C_{frac}. The random step in SRS is rare and negligible for complex microbiome data as noted previously (Beule & Karlovsky, 2020). This rather minor modification, however, ensures the reproducibility of SRS, which is essential for microbiome analysis (Schloss, 2018).

As an example of microbiome count data normalization using SRS, we utilized a bacterial 16S rRNA gene amplicon sequencing dataset comprising 494 samples derived from an ongoing oral microbiome study. The dataset was processed in QIIME 2 (Bolyen et al., 2019). After anonymization of samples and ASVs, an ASV table comprising a random subset of 100 samples was analyzed. The visualization of SRS curves revealed that the observed ASVs did not decay steadily with decreasing number of reads (Figure 2A). This is due to the way how the ranked fractional values (C_{frac}) are picked: depending on the scaling factor, an ASV with an integer value (C_{int}) of zero may or may not be picked by ranked subsampling due to its C_{frac}, causing a reproducible zigzag behaviour in the observed number of species. The magnitude of the zigzag observed in SRS curves depends on the data structure (balance between rare and abundant ASVs). Despite the zigzag behaviour, the observed ASV richness was frequently observed to be higher after SRS as compared to rarefying (Figure 2B). Therefore, we recommend the use of SRS Shiny app (`SRS.shiny.app`-function) prior to SRS for the determination of C_{min} for users working in the R environment. QIIME 2 users are also encouraged to utilize `.qza` files in the SRS Shiny app.

DRAFT

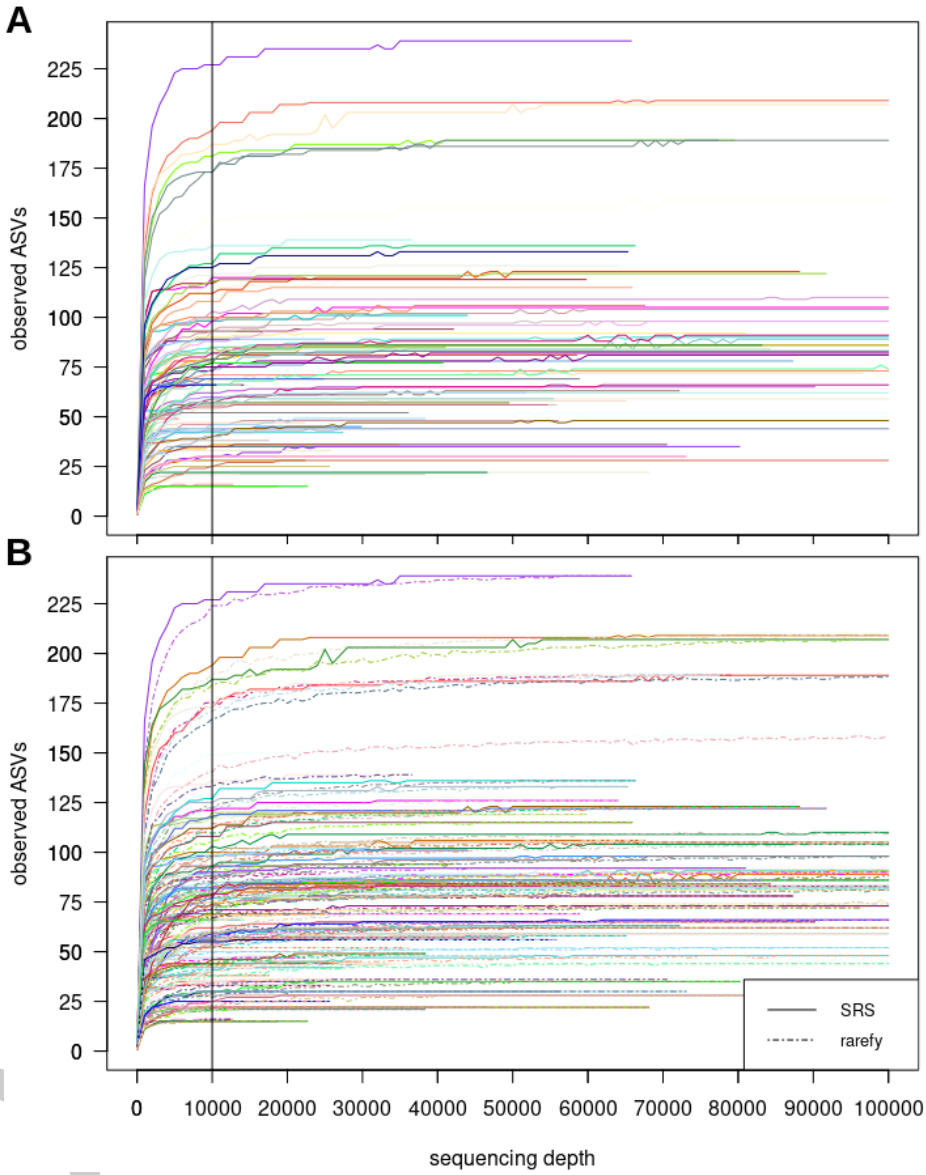

Figure 2: (**A**) SRS curves and (**B**) comparison of SRS curves and repeated rarefying (10 repeats) using the "`richness`" metric (`SRScurve`-function of the 'SRS' R package version 0.2.2). The vertical black solid line indicates the chosen number of counts (10,000) to which all samples will be normalized (C_{min}).

Acknowledgements

The authors thank Devon O'rourke for his suggestions. This work was supported by the German Federal Ministry of Education and Research (BMBF) in the framework of the Bonares-SIGNAL project (funding codes: 031A562A and 031B0510A). V.H. was supported by Fundação de Amparo à Pesquisa do Estado de São Paulo (FAPESP, process no. 13996-0/2018).

References

Beule, L., & Karlovsky, P. (2020). Improved normalization of species count data in ecology by scaling with ranked subsampling (SRS): Application to microbial communities. *PeerJ*, *8*, e9593. https://doi.org/10.7717/peerj.9593

Bolyen, E., Rideout, J. R., Dillon, M. R., Bokulich, N. A., Abnet, C. C., Al-Ghalith, G. A., Alexander, H., Alm, E. J., Arumugam, M., Asnicar, F., & others. (2019). Reproducible, interactive, scalable and extensible microbiome data science using QIIME 2. *Nature Biotechnology*, *37*(8), 852–857. https://doi.org/10.1038/s41587-019-0209-9

Fierer, N. (2017). Embracing the unknown: Disentangling the complexities of the soil microbiome. *Nature Reviews Microbiology*, *15*(10), 579–590. https://doi.org/10.1038/nrmicro.2017.87

McMurdie, P. J., & Holmes, S. (2014). Waste not, want not: Why rarefying microbiome data is inadmissible. *PLoS Computational Biology*, *10*(4), e1003531. https://doi.org/10.1371/journal.pcbi.1003531

Oksanen, J., Blanchet, F. G., Kindt, R., Legendre, P., Minchin, P. R., O'hara, R., Simpson, G. L., Solymos, P., Stevens, M. H. H., Wagner, H., & others. (2013). Package 'vegan.' *Community Ecology Package, Version*, *2*(9), 1–295.

Orsi, W. D. (2018). Ecology and evolution of seafloor and subseafloor microbial communities. *Nature Reviews Microbiology*, *16*(11), 671–683. https://doi.org/10.1038/s41579-018-0046-8

Schloss, P. D. (2018). Identifying and overcoming threats to reproducibility, replicability, robustness, and generalizability in microbiome research. *MBio*, *9*(3), e00525–18. https://doi.org/10.1128/mbio.00525-18

Yatsunenko, T., Rey, F. E., Manary, M. J., Trehan, I., Dominguez-Bello, M. G., Contreras, M., Magris, M., Hidalgo, G., Baldassano, R. N., Anokhin, A. P., & others. (2012). Human gut microbiome viewed across age and geography. *Nature*, *486*(7402), 222–227. https://doi.org/10.1038/nature11053

DRAFT